

SRCS : Systèmes Répartis Client/Serveur

TD 4 : Serveur Multi-threadé

Objectifs pédagogiques

- Serveur multi-threadé
- Pool de threads
- Producteur/consommateur

Le but d'un pool de threads est de limiter le nombre de threads créés par un programme. Dans de nombreux cas (par exemple dans un serveur Web, lorsque de nombreux clients arrivent simultanément), une concurrence débridée et non contrôlée peut amener un écroulement de la machine. Cet exercice a donc pour but d'étudier les mécanismes liés aux pools de threads.

Question 1

Dans un premier temps on considère un **Pool constant** c'est à dire un pool qui contient un nombre constant de threads créés initialement lors de la mise en place du pool. Comment fonctionne ce pool ? Quelle structure de données est nécessaire à son fonctionnement ? A quel " grand problème système " peut-on rattacher la gestion de cette structure de données ?

Question 2

On considère :

- une classe **Job** qui est instanciée à chaque arrivée de client et possède un attribut de type **Socket** représentant la socket de connexion associée à ce client. Cette classe possède une méthode **void traiterJob()** qui définit le code du Job. Cette méthode ferme la socket de connexion une fois le job exécuté.
- une classe **Tampon** fournissant :
 - ◇ une méthode **put** pour déposer un Job
 - ◇ une méthode **get** pour retirer un Job.
- que l'on dispose des primitives de synchronisation **wait()** et **notifyAll()** vues en cours.

Donnez le code Java de la classe **Job** et de la classe **Tampon** permettant la gestion des Job.

Question 3

Donner le code Java de la classe **Worker** qui décrit le comportement d'un thread du pool.

Question 4

Donner le code Java de la fonction principale du serveur. Ilinstanciera le pool de thread, et ajoutera dans le tampon un nouveau **Job** à chaque nouvelle connexion.

Question 5

Lorsqu'on ne connaît pas a priori le nombre de clients attendus, quel problème pose le pool fixe ? Proposer une solution pour y remédier.

Question 6

Dans système à **pool dynamique** pour éviter de créer un thread à chaque arrivée de client, il peut être rentable de toujours disposer de threads en attente. Quel problème introduit-on alors ? Comment le résoudre ?

Question 7

On propose de coder la classe **Supervisor** qui représente l'objet chargé de mettre en œuvre la solution à la question précédente. Cette classe possède comme attribut

- une liste des workers en train de travailler
- une liste des workers en attente d'un job
- les bornes min et max du nombre de workers en attente

- la borne max du nombre de workers total existants.

Nous proposons deux solutions :

- **Supervisor** est un fil d'exécution qui monitore le nombre de threads **Worker** en attente et met à jour lui-même les listes ;
- **Supervisor** n'est qu'un objet partagé parmi les threads **Worker** et ce sont ces derniers qui ont la responsabilité de mettre à jour les listes lorsqu'ils commencent ou finissent de travailler.

Comparer ces deux solutions.

Question 8

Codez la classe **Supervisor** et modifiez les classes précédentes en conséquence.